# UD 1. ADMINISTRACIÓN DE SERVIDORES WEB

## 1. Servidores web

Un servidor WEB es un programa informático que consta de **un intérprete HTTP** el cual se mantiene a la espera de peticiones de clientes y le responde con el contenido según sea solicitado. El cliente, una vez recibido el código, lo interpreta y lo exhibe en pantalla.

**Servidores Web más utilizados**

- **Apache**: Es el servidor más utilizado. Parte de su éxito se debe a que es multiplataforma (puede correr en distintos SO) y a que permite emplear diversos lenguajes en el lado del servidor (PHP, Python y Perl principalmente), así como incorporar características como la compresión de datos, las conexiones seguras y la utilización de URLs amigables. Apache crea hilos y sub-hilos para manejar conexiones adicionales, se dice que está basado en procesos. El administrador puede configurar el servidor para controlar el número máximo de hilos permitidos. Esta configuración varía dependiendo de la memoria disponible en la máquina. Ojo porque demasiados procesos en memoria hace que el rendimiento se vea gravemente disminuido. Usado por empresas importantes como Wikimedia
- **Nginx**: Es un servidor web ligero que funciona en múltiples plataformas (entre las que se encuentran Windows Linux y Mac OS X). Su arquitectura, es diferente al modelo tradicional, de crear una instancia por cada request. NGINX es asíncrono y procesa decenas de miles de conexiones simultáneas en un único proceso compacto y con varios núcleos de CPU. Además se compone de módulos que se incluyen en tiempo de compilación. Eso significa que el usuario descarga el código fuente y selecciona qué módulos quiere utilizar, haciendo más liviano que la competencia. Hay módulos para la conexión a clones de aplicaciones, balanceo de carga, servidor proxy, y otros. No hay módulo para PHP, ya que Nginx puede interpretar código PHP en sí mismo. Es usado por algunos sitios importantes como WordPress, Github, Facebook, Instagram, DropBox
- **Internet Information Services (IIS)** Es el 3er servidor web más usado y cuenta con un buen número de módulos, pero también con el gran handicap de funcionar únicamente en Windows.

**Páginas donde se publican periódicamente los servidores web más utilizados**

- http://trends.builtwith.com/Web-Server
- http://w3techs.com/technologies/overview/web_server/all
- http://news.netcraft.com/archives/2012/08/02/august-2012-web-server-survey.html
- https://w3techs.com/
- http://gs.statcounter.com/

## 2. Servidor web Apache

Un servidor web es un programa que se ejecuta de forma continua en un ordenador (también se utiliza el término para referirse al ordenador que lo ejecuta), se mantiene a la espera de peticiones por parte de un cliente (un navegador de Internet) y contesta a estas peticiones de forma adecuada, sirviendo una página web que será mostrada en el navegador o mostrando el mensaje correspondiente si se detectó algún error.

Uno de los servidores web más populares del mercado y el más utilizado actualmente es Apache, de código abierto y gratuito, disponible para Windows y GNU/Linux, entre otros. Desde 1996, Apache, es el servidor HTTP más usado. Alcanzó su máxima cuota de mercado en 2005 siendo el servidor empleado en el 70% de los sitios web en el mundo, sin embargo ha sufrido un descenso en su cuota de mercado en los últimos años

**Características**

- Multiplataforma es decir corre sobre multitud de plataformas y S.O.
- Ofrece tecnología libre y de código abierto.
- Es configurable y de diseño modular, capaz de extender su funcionalidad, Un módulo es una manera de agrupar ciertas funcionalidades. Una de las principales razones de emplear módulos, es que no toda instalación requiere de las mismas funcionalidades y si fueran incluidas todas las funcionalidades en una versión única de Apache, lo haría sumamente pesado en cuanto a requerimientos de Memoria RAM y espacio en Disco Duro, por esto se opta por crear módulos e incluir en las versiones solo lo necesario.
- Extensible: Trabaja en conjunto con gran cantidad de lenguajes de programación interpretados como PHP, Perl, Java, JSP (Java Server Pages) y otros lenguajes de script, que son el complemento ideal para los sitios web dinámicos.
- Al ser tan popular y utilizado, es posible encontrar gran cantidad de documentos, ejemplos, ayuda y soporte en internet en todos los idiomas.

A continuación se ve la instalación y configuración básica de Apache, ya que en otro capítulo del temario se estudiará Apache con más profundidad.

**Instalación de Apache**

- Desde el terminal

```
sudo apt-get update
```

```
sudo apt-get install apache2
```

```
sudo apt install apache2 (más reciente)
```

- Para ver si está instalado el apache y su versión:

```
apache2 -v
```

- También se puede comprobar viendo que se han creado los directorios de instalación

```
ls /etc/apache2/
```

- Con el siguiente comando (hay que instalarlo) se puede ver el árbol de directorios y ficheros gráficamente.

```
tree directorio
```

- El servidor web apache2, al igual que todos los servicios, dispone de un script de arranque y parada en la carpeta /etc/init.d.

```
/etc/init.d/apache2 stop

/etc/init.d/apache2 start

/etc/init.d/apache2 restart
```

- El siguiente comando muestra cada proceso del sistema y toda la información disponible en un listado completo, (every full y grep para filtrar los servicios apache).

```
ps -ef | grep apache
```

- Ejecutamos `netstat -ptan` para ver por qué puerto escucha. (-p processes, -t solo tcp, -a para visualizar ALL conexiones activas en el sistema, -n Displays active TCP connections,)

**EJERCICIO: Detén el servicio y comprueba que no está a la escucha con el comando que muestra los procesos del sistema**

```
/etc/init.d/apache2 stop
```

**ó**

```
apache2ctl stop
```

**Vuelve a arrancar el servicio y comprueba que funciona otra vez**

**`/etc/init.d/apache2 start`**

**ó**

**`apache2ctl start`**

**OJO, VERSIÓN MÁS RECIENTE Usa systemctl en sistemas modernos con systemd (Ubuntu 16.04+, Debian 8+, CentOS 7+).**

**`sudo systemctl start/stop/restart/reload apache2`**

### Resumen de comandos para gestionar Apache

| Acción | Comando moderno | Comando tradicional | Alternativa ( `service` ) |
|---|---|---|---|
| Iniciar Apache | `sudo systemctl start apache2` | `sudo /etc/init.d/apache2 start` | `sudo service apache2 start` |
| Detener Apache | `sudo systemctl stop apache2` | `sudo /etc/init.d/apache2 stop` | `sudo service apache2 stop` |
| Reiniciar Apache | `sudo systemctl restart apache2` | `sudo /etc/init.d/apache2 restart` | `sudo service apache2 restart` |
| Recargar configuración | `sudo systemctl reload apache2` | `sudo /etc/init.d/apache2 reload` | `sudo service apache2 reload` |
| Ver estado del servicio | `sudo systemctl status apache2` | (no aplica) | `sudo service apache2 status` |
| Control manual (extra) | `sudo apache2ctl start/stop/restart` | `sudo apache2ctl [acción]` | (no aplica directamente) |

- Al instalar apache, se habrá creado la carpeta /var/www . Dentro de la carpeta se encuentra el directorio html conteniendo el archivo **index.html**, que es la página que el servidor sirve por defecto (en versiones anteriores no existía la carpeta html)

**EJERCICIO: Comprobar que desde el navegador http://localhost ó http://127.0.0.1 vemos la página por defecto. Modifica el fichero correspondiente para que al acceder a http://localhost aparezca el mensaje "Servidor Apache tuNombre". Antes de modificar el**

**fichero, haz una copia de seguridad en el mismo directorio por ejemplo llamándole index2.html (recuerda el comando para copiar en linux)**

**Desde terminal:**

```
curl http://localhost
```

**EJERCICIO: Crea dentro de /var/www/html otra carpeta llamada mispaginas (recuerda el comando para crear carpetas en Linux) , y mete dentro una página llamada pag.html que muestre "hola qué tal" respetando las etiquetas html (usa el comando touch para crear la página y luego edítala)**

- **Desde el navegador de la máquina virtual accede a localhost.**
- **Desde el navegador de la máquina virtual accede a localhost/mispaginas**
- **Desde el navegador de la máquina virtual accede a localhost/mispaginas/pag.html**

**<u>Layout de Apache para Debian, Ubuntu (Apache httpd 2.x):</u>**

| | |
|---|---|
| /etc/apache2 | directorio de archivos de configuración de apache2 (ServerRoot) |
| /var/www | DocumentRoot |
| /etc/init.d/ | el archivo apache2 para el arranque, parada del servidor |
| /usr/lib/apache2 | Módulos y librerías de apache2 |
| /usr/sbin | archivos binarios |
| /usr/share/doc/apache2 | documentación |
| /var/log/apache2 | directorio de registro de apache |

# 3. Archivos de configuración Apache 2.4 en distribuciones basadas en Debian (como Ubuntu)

- Desde el terminal sudo apt-get update
- Desde el terminal sudo apt-get install apache2
- Para ver la versión, desde el terminal: apache2 –v

(OJO LA VERSIÓN A UTILIZAR ES LA 2.4 que tiene algunos cambios con respecto la anterior http://www.justgohome.co.uk/blog/2014/04/new-in-14-04-apache.html)

Recordemos que los archivos de configuración de apache2 se encuentran en la carpeta /etc/apache2

```
├── apache2.conf
├── conf-available
    | ├── charset.conf
    | ├── localized-error-pages.conf
    | ├── other-vhosts-access-log.conf
    | ├── security.conf
    | └── serve-cgi-bin.conf
├── conf-enabled
    | ├── charset.conf -> ../conf-available/charset.conf
    | ├── localized-error-pages.conf -> ../conf-available/localized-error-pages.conf
    | ├── other-vhosts-access-log.conf -> ../conf-available/other-vhosts-access-log.conf
    | ├── security.conf -> ../conf-available/security.conf
    | └── serve-cgi-bin.conf -> ../conf-available/serve-cgi-bin.conf
├── envvars
├── magic
├── mods-available
├── mods-enabled
├── ports.conf
├── sites-available
    | ├── 000-default.conf
```

```
|   └── default-ssl.conf
└── sites-enabled
    └── 000-default.conf -> ../sites-available/000-default.conf
```

| | |
|---|---|
| **apache2.conf** | En este archivo se encuentra la configuración del servidor web apache. Se recomienda no modificar aquí sino en el directorio conf-available. |
| **envvars** | Contiene la información del usuario, grupo Y PID del servicio de apache. |
| **mods-available** | Directorio en se depositan los módulos que se agregan al servidor apache. |
| **mods-enabled** | Directorio donde se encuentranlos módulos activados y disponibles |
| **ports.conf** | Archivo de configuración en donde se especifica el puerto de escucha, por default es 80 (http) y 443 (https). |
| **sites-available** | Directorio en donde se encuentran los archivos de configuración de sitios o páginas web que configuremos. |
| **sites-enabled** | Directorio en donde se habilita los sitios web. |
| **conf-available (anterior conf.d)** | Directorio en donde están los archivos de configuración de apache. |
| **magic** | Configuración del módulo mod_mime_magic. Se encarga de transferir ficheros de determinados tipos. |

# 4. Propiedades de las Directivas en Apache

Una vez vistos los ficheros de configuración de Apache, hay que mencionar que en estos ficheros la configuración se realiza a través de **directivas**. Todas las directivas disponibles en la distribución estándar de Apache están en:

http://httpd.apache.org/docs/2.4/es/mod/directives.html

Para describir las directivas de Apache se usan los siguientes términos:

**SINTAXIS**

La sintaxis es específica para cada directiva, y se describe en detalle en la definición de cada una de ellas. Generalmente, es el nombre de la directiva seguido del argumento o argumentos que correspondan separados por espacios. Si un argumento contiene un espacio, entonces debe escribirse entre comillas. Los argumentos opcionales van entre corchetes ([]). Si cada argumento puede tomar más de un valor, los valores posibles van separados por barras verticales "|". Los textos literales (los que no hay que sustituir) están en el tipo de letra por defecto del resto del texto, mientras que los que hay que sustituir están resaltados. Las directivas que pueden tomar un número variable de argumentos terminan con puntos suspensivos ("...").

**VALOR POR DEFECTO**

Si una directiva tiene un valor por defecto y no se especifica un valor explícitamente en la configuración, el servidor Apache se comportará como si hubiera especificado ese valor por defecto.

**CONTEXTO**

Estas **directivas** que tienen posibilidad de ser usadas dentro de un **contexto**, es decir, un ámbito en el que pueden ser aplicadas. Hay cuatro posibilidades donde las directivas pueden ser usadas.

- **Server config**: Este contexto hace referencia a que la directiva puede aparecer dentro de los ficheros de configuración del servidor (por ejemplo apache2.conf u otros) pero no puede usarse ni dentro de un <VirtualHost> ni dentro de <Directory> ni tampoco en ficheros .htaccess.

- **Virtual host**: Este contexto hace referencia a que la directiva puede aparecer dentro de un <VirtualHost> en los ficheros de configuración del servidor.
- **Directory**: En este contexto se incluyen directivas dentro de los contenedores <Directory>, <Location>, <Files>, <If>, y <Proxy> de los ficheros de configuración.
- **.htaccess**: Si una directiva es válida en este contexto, eso significa que puede aparecer en los ficheros .htaccess.

Una directiva puede usarse *solo* en el contexto especificado, si la usa en otro sitio, se producirá en error de configuración que hará que el servidor no pueda servir peticiones en el contexto correctamente, o que el servidor no pueda funcionar, *por ejemplo*, puede que el servidor no se inicie.

Las ubicaciones válidas para una directiva son el resultado de la operación booleana OR de todos los contextos listados más arriba en que esté permitido su uso. En otras palabras, una directiva que esté marcada como válida en "server config, .htaccess" puede usarse tanto en el fichero httpd.conf como en los ficheros .htaccess, pero no dentro de las secciones <Directory> o <VirtualHost>.

**PREVALECE SOBRE (OVERRIDE)**

Este atributo indica qué configuración de las especificadas para una directiva es la que prevalece cuando la directiva aparece en un fichero .htaccess. Si el contexto de una directiva no permite que aparezca en ficheros .htaccess, entonces no aparecerá ningún contexto en este campo.

Para que se aplique el valor especificado en este campo se usa la directiva AllowOverride, y se aplica a un entorno en particular (por ejemplo un directorio) y todo lo que haya por debajo de él, a menos que haya alguna modificación posterior por directivas AllowOverride a niveles más bajos. La documentación de esta directiva también especifica los valores que puede tomar override.

**ESTADO**

Indica el grado de integración con el servidor web Apache que presenta la directiva; en otras palabras, puede que tenga que recompilar el servidor con un conjunto mejorado de módulos para tener acceso a algunas directivas y a sus funcionalidades.

**MÓDULO**

Indica el módulo en el cual se define la directiva.

Ejemplo de directiva con propiedades

**DirectoryIndex Directive**

| | |
|---|---|
| Description: | List of resources to look for when the client requests a directory |
| Syntax: | `DirectoryIndex disabled \| local-url [local-url] ...` |
| Default: | `DirectoryIndex index.html` |
| Context: | server config, virtual host, directory, .htaccess |
| Override: | Indexes |
| Status: | Base |
| Module: | mod_dir |

http://httpd.apache.org/docs/2.4/mod/directive-dict.html

# 5. Configuración global del servidor

El archivo de configuración (etc/apache2/apache2.conf) está compuesto por directivas que le indican al servidor cómo actuar. **IMPORTANTE: Cada vez que haces un cambio en el archivo de configuración es necesario reiniciar el servidor para que surta efecto.**

**CONFIGURACIÓN /etc/apache/apache2.conf**

Se describen algunas directivas del server.

- **ServerRoot**: Indica la ubicación de los ficheros de configuración del servidor Apache.
- **PidFile**: Fichero en el que se almacenará el número de proceso con el que se está ejecutando el servidor Apache. No se indica un directorio o ubicación sino que se le asigna el valor de la variable "APACHE_PID_FILE" la cual está definida en el archivo de configuración "envvars". Su valor puede ser útil cuando otros procesos quieren mandarle una señal. Por ejemplo en caso de querer matarlo kill -9 <pid>
- **Directivas relacionadas con el tiempo de espera de las conexiones con los clientes.**

```
Timeout 300

KeepAlive On

MaxKeepAliveRequests 100

KeepAliveTimeout 15
```

  - Timeout: Indica el número de segundos antes de que se cancele una conexión por falta de respuesta.
  - KeepAlive On: Indica si se permiten o no las conexiones persistentes, es decir más de una petición por conexión. Por ejemplo servirá para servidores donde se tenga que despachar páginas + imágenes por ejemplo, de lo contrario mantenerlo en Off.
  - MaxKeepAliveRequests: Indica el máximo número de peticiones que se permiten en conexiones persistentes. Un valor 0 permite un número ilimitado. Se recomienda dejar este valor elevado para obtener un mayor rendimiento.
  - KeepAliveTimeout indica el número de segundos de espera para la siguiente petición del mismo cliente con la misma conexión.

- **Usuario y grupo**

```
User ${APACHE_RUN_USER}

Group ${APACHE_RUN_GROUP}
```

Aunque nosotros hemos instalado el servidor Apache como administradores (root), la aplicación no podrá ejecutarse con estos privilegios porque provocaría muchos problemas de seguridad. En lugar de ello, el servidor Apache dispondrá de su propio usuario y grupo con el que se ejecutará. De nuevo hace referencia a unos valores almacenados en las variables APACHE_RUN_USER Y APACHE_RUN_GROUP que se podrá localizar en el archivo "envvars".

- **HostnameLookups Off:** HostnameLookups registra los nombres o las direcciones IP de los clientes. Por defecto está en Off para evitar que para cada consulta se tenga que realizar una resolución DNS para averiguar el nombre y disminuya el rendimiento.
- **ErrorLog:** dónde ubicar el archivo de registro de los errores que se produzcan en el servidor
- **LogLevel warn:** Controla el número de mensajes registrados en error_log. Puede ser: debug, info, notice, warn, error, crit, alert, emerg.
- **LogFormat:** permite definir distintos nombres para modelos de log que queremos guardar o registrar sobre las consultas que se hacen al servidor. Apache viene con algunos formatos definidos combined, common …

  Por ejemplo

```
LogFormat "%h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\"" combined
```

  ¿Qué significa cada argumento?

  http://httpd.apache.org/docs/2.4/mod/mod_log_config.html
- **Directory:** Esta opción se usa para configurar cómo se comportará y qué se permitirá en cada directorio al que tiene acceso el servidor Apache. Esta configuración se aplica a un directorio y los subdirectorios que contiene si no se sobreescribe en otra definición sobre un directorio más concreto. Nos encontramos TRES veces esta etiqueta. La primera hace referencia al directorio raíz y se configura siempre con

opciones muy restrictivas. Los subdirectorios heredan la configuración de los directorios padres y sobreescriben su configuración.

```
<Directory />
      Options FollowSymLinks
      AllowOverride None
      Require all denied
</Directory>

<Directory /usr/share>
      AllowOverride None
      Require all granted
</Directory>

<Directory /var/www/>
      Options Indexes FollowSymLinks
      AllowOverride None
      Require all granted
</Directory>
```

Opciones sobre directorios:

`<Directory /directorio>` se usa para configurar ciertos directorios del servidor

- Por defecto, para el directorio raíz `<Directory />` *(*fíjate que la barra invertida señala el directorio raíz), no hay características permitidas salvo las siguientes:
  - `Options FollowSymLinks`: La directiva *Options* controla características del servidor indicando con el atributo *FollowSymLinks* que el servidor puede seguir enlaces simbólicos en el directorio raíz.
  - `AllowOverride`: Establece qué directivas están permitidas en un fichero .htaccess. Por defecto, el directorio raíz está configurado para no permitir la prevalencia del fichero .htaccess *(None).*Es decir el fichero htaccess podría sobreescribir valores establecidos por defecto en el servidor si está activado, con None no se puede sobreescribir ningún valor.

    https://httpd.apache.org/docs/2.4/es/mod/core.html#allowoverride

**AllowOverride Directiva**

| | |
|---|---|
| Descripción: | Types of directives that are allowed in `.htaccess` files |
| Sintaxis: | `AllowOverride All\|None\|directive-type [directive-type] ...` |
| Valor por defecto: | `AllowOverride None (2.3.9 and later), AllowOverride All (2.3.8 and earlier)` |
| Contexto: | directory |
| Estado: | Core |
| Módulo: | core |

- Cualquier directorio del sistema que necesite valores más permisivos ha de ser configurado explícitamente como ocurre con el *<Directory/var/www>.* Por defecto, en el directorio DocumentRoot ( /var/www), *Options* está configurado para incluir los comandos *Indexes*, *FollowSymLinks*
  - `Indexes` como ya se ha comentado anteriormente permite al servidor generar un listado del directorio.
  - `FollowSymLinks` permite al servidor seguir enlaces simbólicos en ese directorio.

  http://httpd.apache.org/docs/2.4/es/mod/core.html#options

Normally, if multiple `Options` could apply to a directory, then the most specific one is used and others are ignored; the options are not merged. (See how sections are merged.) However if *all* the options on the `Options` directive are preceded by a + or - symbol, the options are merged. Any options preceded by a + are added to the options currently in force, and any options preceded by a - are removed from the options currently in force.

**Warning**
Mixing `Options` with a + or - with those without is not valid syntax, and is likely to cause unexpected results.

For example, without any + and - symbols:

```
<Directory /web/docs>
   Options Indexes FollowSymLinks
</Directory>

<Directory /web/docs/spec>
   Options Includes
</Directory>
```

then only `Includes` will be set for the `/web/docs/spec` directory. However if the second `Options` directive uses the + and - symbols:

```
<Directory /web/docs>
   Options Indexes FollowSymLinks
</Directory>

<Directory /web/docs/spec>
   Options +Includes -Indexes
</Directory>
```

then the options `FollowSymLinks` and `Includes` are set for the `/web/docs/spec` directory.

  - `Require all granted`: Acceso permitido incondicionalmente. Ver: http://httpd.apache.org/docs/2.4/mod/mod_authz_core.html#require
    Sustituye a las anteriores directivas (Order, Allow, Deny) de apache 2.2 NO USARLAS

- **AccessFileName**

Indica el nombre (".htaccess") que deberá tener el fichero local de un directorio en el que se almacenen directivas de acceso.

```
AccessFileName .htaccess
```

.htaccess es un archivo de texto oculto (por esa razón comienza por ".") que permite personalizar el funcionamiento del servidor Apache un directorio concreto sin necesidad de modificar el fichero de configuración principal "apache2.conf".

Cuando un cliente web solicita un archivo al servidor, este busca este archivo desde el directorio raíz hasta el subdirectorio que contiene el archivo solicitado y tiene en cuenta las directivas contenidas en el mismo antes de proceder con la petición.

Los usos principales del .htaccess son:

- Impedir el listado de los archivos de un directorio.
- Redireccionar el tráfico Web.
- Personalizar las páginas de error.
- Restringir el acceso a determinados archivos.
- Impedir el acceso a determinadas IPs o rangos de IP.

- **Include y IncludeOptional:** Incluye los ficheros de configuración de los directorios (OJO ha cambiado de la versión anterior)

<u>Módulos</u>

Recuerda que Apache es de los servidores Web más usados por su modularidad. Es decir se pueden agregar funcionalidades al servidor por medio de módulos. Apache por defecto incluirá todos los módulos existentes en el directorio "/etc/apache2/mods-enabled/". Todos estos ficheros son funcionalidades extra del servidor.

```
# Include module configuration:
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf
```

Fíjate que los archivos que aparecen dentro de la carpeta "mods-enabled" son enlaces a la carpeta "mods-available".

Puerto

Incluirá las directivas incluidas en el archivo “ports.conf”, que hacen referencia al puerto por el que se dará servicio (normalmente el puerto 80).

```
Include ports.conf
```

En ese fichero aparece la directiva Listen que especifica el puerto (y, opcionalmente, la dirección IP) por el que escuchará Apache2.

Si no se especifica la dirección IP, Apache2 escuchará por todas las direcciones IP asignadas a la máquina en la que se ejecute.

El valor predeterminado de la directiva Listen es 80. Cambiarlo a 127.0.0.1:80 provoca que Apache2 sólo escuche por su dispositivo loopback, de forma que no estará disponible para Internet.

**- EJERCICIO: Cámbialo a 81 para cambiar el puerto por el que escucha, y prueba que funciona accediendo desde un cliente. Vuelvelo a dejar como estaba.**
**OJO con lo que indica el fichero ports.conf**
**# If you just change the port or add more ports here, you will likely also**
**# have to change the VirtualHost statement in**
**# /etc/apache2/sites-enabled/000-default.conf**

Dominios virtuales

Incluye el directorio “/etc/apache2/sites-enabled”, que es el lugar donde se definirán los distintos dominios virtuales que deseemos ofrecer con nuestro servidor.

```
# Include the virtual host configurations:
IncludeOptional sites-enabled/*.conf
```

¿Qué dos sitios se han definido por defecto?

Fíjate que los archivos que aparecen dentro de la carpeta “sites-enabled” son enlaces a la carpeta “sites-available”.

Ficheros de configuración

```
# Include generic snippets of statements
IncludeOptional conf-enabled/*.conf
```

**Otras directivas que se pueden configurar a nivel de servidor global (se recomienda configurarlas dentro del directorio conf-available en vez de en apache2.conf)**

- **ServerName:** Nombre del servidor. ServerName localhost

  Ojo, Apache lanza un warning si esta directiva no está configurada a nivel global del server

  *AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message*

  Para eliminar este warning tenemos dos opciones (mejor la 2):

  1) Meter en apache2.conf la directiva
     `ServerName localhost`
  2) Utilizar el directorio conf-available creando el fichero “servername.conf” con dicha directiva y activar la configuración creando el link simbólico a conf-enabled con el comando
     `sudo a2enconf servername`

- **ServerAdmin:** Esta opción permite configurar la dirección del administrador del servidor web que se mostrará si el servidor genera una página de error.

teresa@daw.enlaces

## Otras directivas

- **Códigos de error (ErrorDocument)**

Códigos HTTP Apache: http://members.tripod.com/r_castaneda0/

Se pueden configurar los mensajes de los errores que lanza Apache, para indicar qué deben mostrar los errores a nivel global del servidor en /etc/apache2/conf-available/localized-error-pages.conf , o también a nivel de Virtual host…

```
SINTAXIS: ErrorDocument <3-digit-code> <action>
```

donde action puede ser:

1. Texto a mostrar. Ponga antes del texto que quiere que se muestre unas comillas ("). Lo que sea que siga a las comillas se mostrará. Nota: las comillas (") no se muestran.
2. Una URL local a la que se redireccionará la petición.
3. Una URL externa a la que se redireccionará la petición.

```
ErrorDocument 404 "Lo siento esa pagina no existe"
```

**Buscar directivas en un directorio**

Si quieres buscar en qué ficheros está una directiva definida se puede usar grep –lir directiva directorio, por ejemplo:

```
grep -lir documentRoot  "/etc/apache2/"
```

**EJERCICIO: Se pide añadir la configuración necesaria que me permita añadir una página de error personalizado cuando el server devuelva un 404 a nivel general del servidor tal y como muestra la siguiente imagen. Para ello modifica el fichero /etc/apache2/conf-available/localized-error-pages.conf con las directivas necesarias. Puedes colocar el html personalizado en "/usr/share/apache2/error". Ojo! Si defines una ruta con barra diagonal (/) al principio, Apache lo interpreta como una ruta relativa a la raíz del servidor web (no al sistema de archivos). Si quieres usar una ruta absoluta del sistema de archivos (por ejemplo, un archivo fuera del directorio web, como en /usr/share), Apache no lo puede hacer directamente desde la directiva ErrorDocument. Necesitas mapear la ruta completa del sistema de archivos a una URL accesible para Apache, normalmente usando la directiva alias.**

**<u>Ejemplo:</u>**

**ErrorDocument 404 /error/404.html**

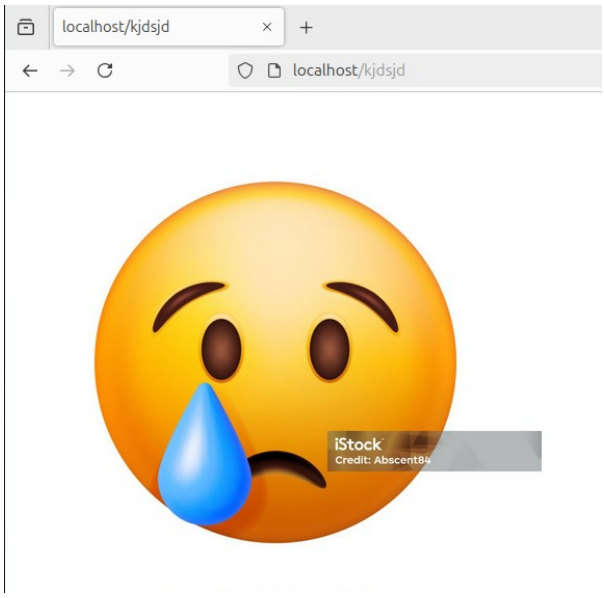
**Apache buscaría ese archivo en el directorio <u>raíz de tu servidor web</u>, generalmente /var/www/html.**

**Solución:**

**Alias /error /usr/share/apache2/error**

**ErrorDocument 404 /error/404.html**

**Ejemplo:**



**EJERCICIO: Queremos hacer una redirección de un recurso a otro en el servidor. Más concretamente se quiere redirigir "**http://localhost/paginas**" a** http://localhost/mispaginas**. Para ello puedes usar la directiva Redirect**

**Redirect "/paginas" "/mispaginas"**

**Crea un fichero de configuración nuevo en /etc/apache2/conf-available/miconf.conf y añade la directiva. Recuerda que siempre que creas una configuración nueva debes activarla con el comando a2enconf. Luego Recarga o reinicia Apache**

**EJERCICIO: Desactivar el listado de archivos en directorios sin index.html, Es decir evitar que Apache muestre el contenido de un directorio si no existe un archivo index.html, mostrando en su lugar un error 403 Forbidden. Para ello crea un fichero de configuración nuevo en /etc/apache2/conf-available/nodirlisting.conf. Añade la directiva adecuada para desactivar los listados de directorios. Puedes aplicarlo globalmente o solo a /var/www/html**

```
<Directory /var/www/html>
   Options -Indexes
</Directory>
```

**Recuerda que siempre que creas una configuración nueva debes activarla con el comando a2enconf. Luego Recarga o reinicia Apache**

# 6. Configuración del host virtual por defecto

Como ya hemos visto, los distintos dominios virtuales que deseemos ofrecer con nuestro servidor se configuran en sites-available. El sitio virtual por defecto que trae Apache está configurado en el fichero /etc/apache2/sites-available/000-default.conf y se observa que básicamente se traduce a:

```
  GNU nano 7.2                                                    /etc/apache2/sites-
available/000-default.conf
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com

        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/html

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

A continuación se detallan algunas directivas que se pueden configurar a nivel de host virtual, localízalas si existen en el host virtual por defecto.

- VirtualHost *:80 Esta directiva define el host virtual, indicando la dirección IP donde va a responder el servidor (podríamos tener una tarjeta de red con varias direcciones IP) y el puerto (80 es el puerto por defecto donde escucha Apache). Si

queremos que escuche en todas las direcciones IP del servidor colocamos un asterisco quedaría *:80.

- *ServerName* es el nombre del host virtual.
- *DocumentRoot* path donde están guardados los archivos de nuestra página, puede ser cualquier carpeta donde tengamos los ficheros (cuidado con la seguridad y los permisos de la carpeta).
- *ServerAdmin* especifica la dirección de correo del administrador del servidor. El valor por omisión es webmaster@localhost. Esta dirección se utiliza para que si tu sitio web tiene algún problema, Apache2 muestre un mensaje de error en el que aparecerá esta dirección de correo para que la gente pueda enviar un informe del error.
- *CustomLog* fichero log donde el servidor escribe los registros de las transferencias.
- *ErrorLog* archivo log en el que se registrarán los errores.

  Estos dos ficheros se mantienen separados para ayudar en la resolución de problemas.

## Consideraciones importantes sobre los permisos del DocumentRoot

Cuando instalas Apache en una distribución como **Ubuntu** o **Debian**, los permisos por defecto para el directorio DocumentRoot (como /var/www/html) suelen ser establecidos de la siguiente manera:

**Ejemplo de permisos después de la instalación:**

drwxr-xr-x 3 root root 4096 sep 24 2024 /var/www/html

Esto significa lo siguiente:

- **drwxr-xr-x**: Son los permisos del directorio:
  - **d**: Es un directorio.
  - **rwx** (propietario): El usuario root tiene **lectura**, **escritura** y **ejecución** (acceso completo).
  - **r-x** (grupo): El grupo root tiene **lectura** y **ejecución** (puede acceder y leer el contenido, pero no modificarlo).
  - **r-x** (otros): Los demás usuarios también tienen **lectura** y **ejecución** (pueden ver y acceder al directorio, pero no modificarlo).

Apache generalmente se ejecuta bajo el usuario **www-data**. Esto significa que **Apache necesitará permisos de lectura sobre los archivos** dentro de DocumentRoot.

¿El usuario www-data necesita permisos de escritura? Si **Apache** necesita **escribir** en el DocumentRoot (por ejemplo, para guardar archivos subidos), los permisos actuales (root como propietario y root como grupo) no permiten que Apache (que corre como www-data) escriba allí. Esto podría generar errores o limitaciones en ciertas funcionalidades de tu servidor web.

Recomendaciones para permisos adecuados:

Como Apache ejecuta el servicio bajo el usuario www-data, es una buena práctica cambiar la propiedad del directorio DocumentRoot a www-data, para que Apache pueda **leer y escribir** los archivos cuando sea necesario.

```
sudo chown -R www-data:www-data /var/www/html
```

Esto cambia el propietario y grupo de los archivos a www-data, que es el usuario bajo el que Apache corre.

Después de cambiar el propietario, ajusta los permisos para que Apache tenga acceso adecuado:

- **Directorios**: Asegúrate de que los directorios sean **ejecutables** para que Apache pueda navegar a través de ellos. Generalmente, **755** es suficiente.

```
sudo find /var/www/html -type d -exec chmod 755 {} \;
```

- **Archivos**: Los archivos deberían ser **solo de lectura** para Apache (y escritura solo para el propietario si es necesario). Usualmente, **644** es adecuado.

```
sudo find /var/www/html -type f -exec chmod 644 {} \;
```

**Ojo si el directorio tiene permisos demasiado amplios (por ejemplo, 777), cualquier usuario podría modificar los archivos, lo cual es un riesgo de seguridad grave.**

# 7. Secciones de configuración (Contextos de Apache)

Ya hemos estudiado que las directivas presentes en los ficheros de configuración pueden ser de aplicación para todo el servidor, o puede que su aplicación se limite solamente a determinados directorios, ficheros, hosts, o URLs.

Hay cuatro posibilidades donde las directivas pueden ser usadas.

- **Server config**:
- **Virtual host**:
- **Directory**:
- **.htaccess**:

¿Pero en qué orden se aplican? El orden en que se interpretan las directivas según dónde estén colocadas es el siguiente. Algunas reglas son las siguientes:

1) Las directivas de .htaccess prevalecen sobre Directory
2) Directory: Si aparecen varios `<Directory>`, se procesan en el orden en que aparezca en los ficheros de configuración. Si afectan al mismo directorio se procesa empezando por los componentes de la ruta al directorio más corto. Por ejemplo, `<Directory /var/web/dir>` se procesará antes de `<Directory /var/web/dir/subdir>`
3) Las secciones incluidas dentro de secciones <VirtualHost> se aplican *después de* las correspondientes secciones fuera de la definición del host virtual. Esto permite que la configuración especificada para los hosts virtuales pueda prevalecer sobre la configuración del servidor principal.
4) Las secciones que aparecen después prevalecen sobre las que aparecen antes.
5) Las configuraciones incluidas mediante la directiva Include se tratarán como si estuvieran dentro del fichero de configuración principal en lugar de la sección Include.
6) El orden de procesado de las siguientes directivas es
    a. `<Directory>`
    b. `<DirectoryMatch>`
    c. `<Files>` y `<FilesMatch>`
    d. `<Location>` y `<LocationMatch>`

    Por lo que las últimas prevalecen sobre las anteriores

http://httpd.apache.org/docs/2.4/es/sections.html